

Database Management System (DBMS)

BAB 5 — SQL Manipulasi Data (DML)

1. Pengertian DML

DML (*Data Manipulation Language*) adalah bagian dari SQL yang digunakan untuk mengelola isi data dalam tabel.

Operasi utama:

INSERT

SELECT

UPDATE

DELETE

2. INSERT

Digunakan untuk menambahkan data baru ke dalam tabel.

Sintaks:

```
INSERT INTO nama_tabel  
VALUES (...);
```

Contoh:

```
INSERT INTO mahasiswa  
VALUES ('101','Andi','Informatika');
```

3. INSERT dengan Nama Kolom

Lebih aman karena kolom dituliskan secara eksplisit.

```
INSERT INTO mahasiswa  
(nim,nama,prodi)  
VALUES  
('102','Budi','Sistem Informasi');
```

4. Menambahkan Banyak Data Sekaligus

```
INSERT INTO mahasiswa  
(nim,nama,prodi)  
VALUES  
('103','Caca','Informatika'),  
('104','Dina','Sistem Informasi'),  
('105','Eko','Teknik Komputer');
```

5. SELECT

Digunakan untuk mengambil data dari tabel.

Sintaks:

```
SELECT kolom  
FROM tabel;
```

6. Menampilkan Seluruh Kolom

```
SELECT *  
FROM mahasiswa;
```

Tanda:

*

berarti semua kolom.

7. Menampilkan Kolom Tertentu

```
SELECT nama, prodi  
FROM mahasiswa;
```

Hanya menampilkan:

nama

prodi

8. Alias Kolom

Alias digunakan untuk mengganti nama tampilan kolom.

```
SELECT nama AS Nama_Mahasiswa  
FROM mahasiswa;
```

9. DISTINCT

Menghilangkan data duplikat.

Contoh:

```
SELECT DISTINCT prodi  
FROM mahasiswa;
```

10. WHERE

Digunakan untuk melakukan filtering data.

Sintaks:

```
SELECT *  
FROM mahasiswa  
WHERE kondisi;
```

11. Operator Perbandingan

=

>

<

>=

<=

<>

Contoh:

```
SELECT *  
FROM mahasiswa  
WHERE ipk > 3.5;
```

12. Operator AND

Semua kondisi harus bernilai benar.

```
SELECT *  
FROM mahasiswa  
WHERE prodi='Informatika'  
AND ipk > 3.5;
```

13. Operator OR

Minimal satu kondisi benar.

```
SELECT *  
FROM mahasiswa  
WHERE prodi='Informatika'  
OR prodi='Sistem Informasi';
```

14. Operator NOT

Membalik kondisi.

```
SELECT *  
FROM mahasiswa  
WHERE NOT prodi='Informatika';
```

15. BETWEEN

Memilih data dalam rentang tertentu.

```
SELECT *  
FROM mahasiswa  
WHERE ipk BETWEEN 3.0 AND 4.0;
```

16. IN

Memilih data berdasarkan beberapa nilai.

```
SELECT *  
FROM mahasiswa  
WHERE prodi IN  
('Informatika','Sistem Informasi');
```

17. LIKE

Digunakan untuk pencarian pola.

Karakter khusus:

%

—

18. LIKE dengan %

```
SELECT *  
FROM mahasiswa  
WHERE nama LIKE 'A%';
```

Artinya:

Nama diawali huruf A

19. LIKE dengan _

```
SELECT *  
FROM mahasiswa  
WHERE nama LIKE '_ni';
```

Artinya:

Tiga huruf

Berakhiran "ni"

20. ORDER BY

Mengurutkan data.

Urut naik:

```
SELECT *  
FROM mahasiswa  
ORDER BY nama ASC;
```

21. Urut Turun

```
SELECT *  
FROM mahasiswa  
ORDER BY ipk DESC;
```

22. LIMIT

Membatasi jumlah data yang ditampilkan.

```
SELECT *  
FROM mahasiswa  
LIMIT 5;
```

23. UPDATE

Digunakan untuk memperbarui data.

Sintaks:

```
UPDATE tabel  
SET kolom = nilai  
WHERE kondisi;
```

24. Contoh UPDATE

```
UPDATE mahasiswa  
SET prodi='Informatika'  
WHERE nim='101';
```

25. UPDATE Banyak Kolom

```
UPDATE mahasiswa  
SET nama='Andi Saputra',  
    prodi='SI'  
WHERE nim='101';
```

26. Bahaya UPDATE Tanpa WHERE

Query:

```
UPDATE mahasiswa  
SET prodi='IF';
```

Akibat:

Seluruh baris berubah

27. DELETE

Digunakan untuk menghapus data.

Sintaks:

```
DELETE FROM tabel  
WHERE kondisi;
```

28. Contoh DELETE

```
DELETE FROM mahasiswa  
WHERE nim='101';
```

29. DELETE Tanpa WHERE

```
DELETE FROM mahasiswa;
```

Akibat:

Semua data terhapus

Struktur tabel tetap ada.

30. COUNT()

Menghitung jumlah data.

```
SELECT COUNT(*)  
FROM mahasiswa;
```

31. SUM()

Menjumlahkan data numerik.

```
SELECT SUM(sks)  
FROM mata_kuliah;
```

32. AVG()

Menghitung rata-rata.

```
SELECT AVG(ipk)
FROM mahasiswa;
```

33. MAX()

Mencari nilai terbesar.

```
SELECT MAX(ipk)
FROM mahasiswa;
```

34. MIN()

Mencari nilai terkecil.

```
SELECT MIN(ipk)
FROM mahasiswa;
```

35. GROUP BY

Mengelompokkan data.

```
SELECT prodi,
COUNT(*)
FROM mahasiswa
GROUP BY prodi;
```

36. HAVING

Filter hasil GROUP BY.

```
SELECT prodi,  
COUNT(*)  
FROM mahasiswa  
GROUP BY prodi  
HAVING COUNT(*) > 50;
```

37. Urutan Eksekusi Query

Secara konsep:

FROM

WHERE

GROUP BY

HAVING

SELECT

ORDER BY

LIMIT

38. Contoh Query Lengkap

```
SELECT prodi,  
AVG(ipk)  
FROM mahasiswa  
WHERE angkatan = 2024
```

```
GROUP BY prodi  
HAVING AVG(ipk) > 3.25  
ORDER BY AVG(ipk) DESC;
```

39. Kesalahan Umum

UPDATE tanpa WHERE

Mengubah seluruh data.

DELETE tanpa WHERE

Menghapus seluruh data.

Salah Menggunakan LIKE

```
LIKE 'A'
```

bukan:

```
Diawali A
```

Untuk diawali A:

```
LIKE 'A%'
```

Menggunakan WHERE untuk Fungsi Agregat

Salah:

```
WHERE COUNT(*) > 5
```

Benar:

```
HAVING COUNT(*) > 5
```

40. Hubungan DML dengan DDL

DDL:

CREATE TABLE

ALTER TABLE

DROP TABLE

mengatur struktur.

DML:

INSERT

SELECT

UPDATE

DELETE

mengatur isi data.

Soal Bab 5

Soal 1

Buat query untuk menambahkan data berikut ke tabel Mahasiswa:

NIM = 23001

Nama = Andi

Prodi = Informatika

IPK = 3.75

Soal 2

Tampilkan seluruh data dari tabel Mahasiswa.

Soal 3

Tampilkan hanya kolom:

Nama

IPK

dari tabel Mahasiswa.

Soal 4

Tampilkan seluruh mahasiswa yang memiliki:

IPK > 3.50

Soal 5

Tampilkan seluruh mahasiswa yang berasal dari:

Informatika

atau

Sistem Informasi
menggunakan operator IN.

Soal 6

Tampilkan mahasiswa dengan nama yang diawali huruf:

A

Soal 7

Urutkan data mahasiswa berdasarkan:

IPK tertinggi ke terendah

Soal 8

Perbarui data mahasiswa dengan:

NIM = 23001

menjadi:

Nama = Andi Saputra

Soal 9

Hapus data mahasiswa dengan:

NIM = 23001

Soal 10

Hitung:

Jumlah mahasiswa

Rata-rata IPK

IPK tertinggi

IPK terendah

menggunakan fungsi agregat SQL.

BAB 5 selesai.

Selanjutnya: BAB 6 — SQL Query Lanjutan.